

Project Design Phase-II

Data Flow Diagram & User Stories

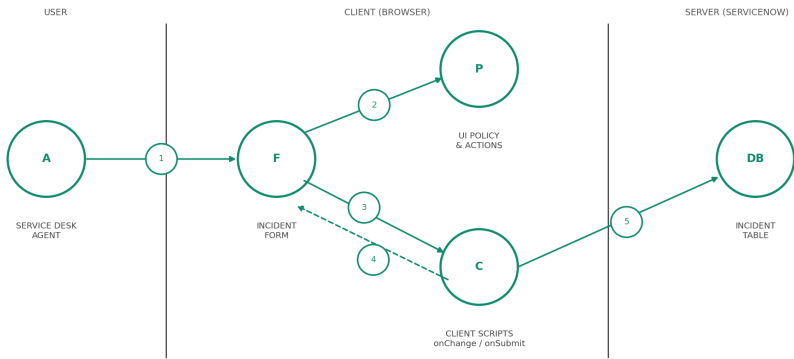
Date	01 October 2026
Team ID	SWTID-2026-5775
Project Name	Implement Client Script & UI Policy (Incident)
Maximum Marks	4 Marks

Data Flow Diagrams:

A Data Flow Diagram (DFD) is a traditional visual representation of the information flows within a system. A neat and clear DFD can depict the right amount of the system requirement graphically. It shows how data enters and leaves the system, what changes the information, and where data is stored. For this project, the DFD shows how the Incident form, UI Policies and Client Scripts exchange data with the service desk agent, the resolver group and the Incident table.

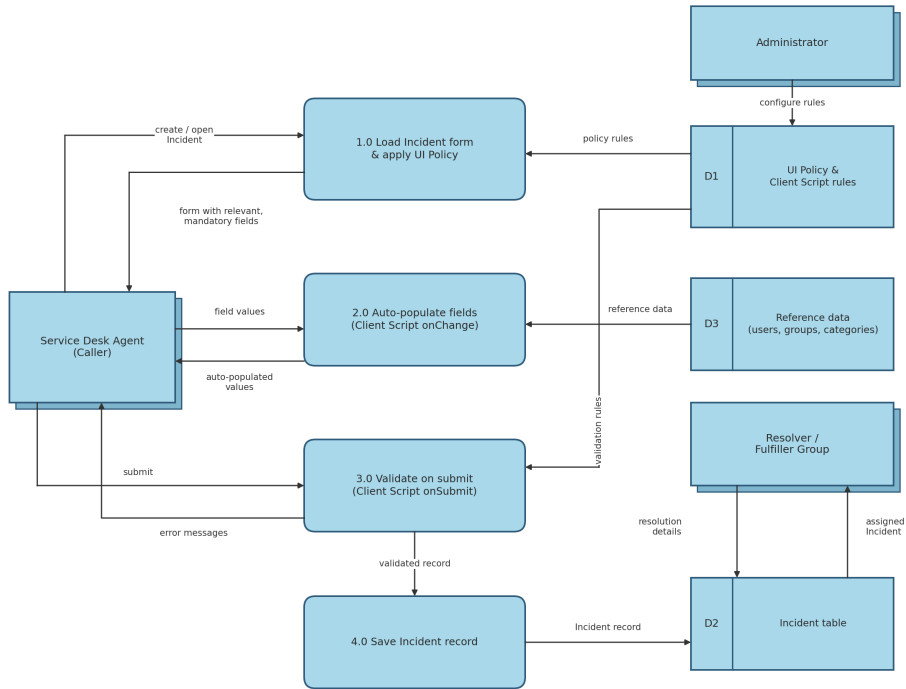
Example: [\(Simplified\)](#)

Flow



1. The service desk agent opens a new Incident form in ServiceNow.
2. UI Policies and UI Policy Actions set fields visible, mandatory or read-only.
3. Client Scripts (onChange) auto-populate dependent fields as values change.
4. On Submit, a Client Script (onSubmit) validates the business rules and returns messages.
5. A valid record is saved to the Incident table; an invalid one is blocked with an error message.

Example: DFD Level 0 (Industry Standard)



User Stories

Use the below template to list all the user stories for the product.

User Type	Functional Requirement (Epic)	User Story Number	User Story / Task	Acceptance criteria	Priority	Release
Service Desk Agent	Conditional Field Behavior (UI Policy)	USN-1	As a service desk agent, I can see only the fields relevant to the selected State so the Incident form is easy to complete.	Fields such as On-hold reason appear only when State is On Hold	High	Sprint-1
		USN-2	As a service desk agent, I am required to fill in fields that become mandatory based on the conditions I select.	Mandatory fields are marked and the record cannot be saved while they are empty	High	Sprint-1
		USN-3	As a service desk agent, I cannot edit certain fields once an Incident is closed.	Closed Incident fields are read-only	Medium	Sprint-2
	Auto-population (Client Script)	USN-4	As a service desk agent, I want dependent fields updated automatically when I change a key field such as Category.	Dependent field values update immediately on change	Medium	Sprint-2
	Save-time Validation (Client Script)	USN-5	As a service desk agent, I receive a clear error message and the Incident is not saved when business rules are not met.	onSubmit script stops the save and shows an error message	High	Sprint-1
		USN-6	As a service desk agent, I see field messages that tell me how to correct an invalid value.	A message appears next to the field with correction guidance	Medium	Sprint-2
Resolver / Fulfiller	Resolution Data Quality	USN-7	As a resolver, I must enter resolution code and close notes before I can resolve an Incident.	Resolution fields are visible and mandatory when State is Resolved	High	Sprint-1
		USN-8	As a resolver, I receive Incidents with a complete short description and category so I can triage quickly.	No new Incident is saved with a blank short description or category	Medium	Sprint-2
Administrator	Rule Configuration	USN-9	As an administrator, I can create UI Policies with conditions and UI Policy Actions without scripting.	Policy is active on the Incident form and its conditions evaluate correctly	High	Sprint-1
		USN-10	As an administrator, I can add onChange and onSubmit Client Scripts for logic a UI Policy cannot handle.	Scripts run on the Incident form without console errors	Medium	Sprint-2
	Testing & Verification	USN-11	As an administrator, I can test every rule with positive and negative cases before release.	All test cases pass and results are documented	Medium	Sprint-3
Process Owner	Reporting Accuracy	USN-12	As a process owner, I want consistent Incident data so SLA tracking and reports are accurate.	Reports show no blank mandatory fields on new Incidents	Medium	Sprint-3